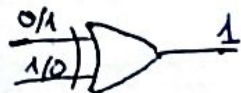


FC: PARCIAL 3: TH7, TH8 y TH9

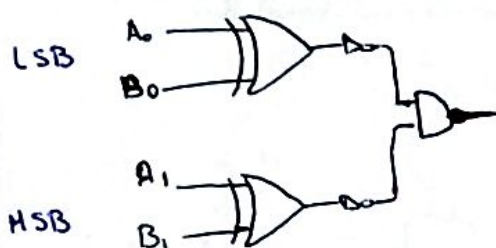
Los circuitos combinatoriales son aquellos cuyo valor de salida dependen exclusivamente de los valores de entrada

- Objetivo: comparar la magnitudes de 2 cantidades binarias para determinar la relación entre ellas. Para ello podemos usar puertas XOR.



Con esta manera
Determinamos si 2
números son iguales o no

- IGUALDAD ENTRE MAGNITUDES



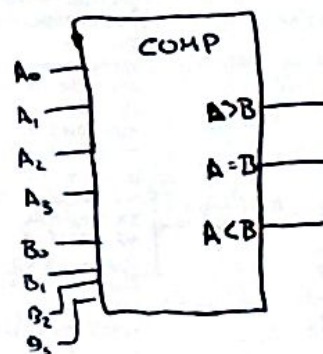
En este ejemplo este circuito compara la igualdad de números binarios de 2 bits ($A = A_1A_0$ y $B = B_1B_0$). Si $A = B$ las 2 comparaciones darán como resultado que son iguales (1). Si $A \neq B$ alguna de las comparaciones dará distinto, dando como resultado que son diferentes (0).

- DESIGUALDAD ENTRE MAGNITUDES

Ahora extendemos la funcionalidad para añadir relaciones ($<$, $>$, $=$).

- Procedimiento: Buscar una desigualdad partiendo desde el MSB. Pues en ese caso:

- Si $A_i = 1$ y $B_i = 0 \rightarrow A > B$
- Si $A_i = 0$ y $B_i = 1 \rightarrow A < B$
- Si no hay desigualdad $\rightarrow A = B$



El DISEÑO CIRCUITO COMPARADOR:

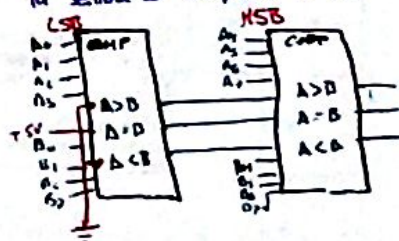
Partimos del diseño de un módulo comparador básico de 2 n° X e Y . El comparador básico necesita 2 salidas: G y L .

VALORES SEÑALES	FUNCIÓN
$G = 1$	$X > Y$
$G = 0$	$X \leq Y$
$L = 1$	$X < Y$
$L = 0$	$X \geq Y$
$G = 0$ $L = 0$	$X = Y$

- Procedimiento:

- Hacemos la tabla en base al n° de bits
- Realizamos Karnaugh para G y L
- Creamos el circuito

Se puede realizar la conexión en cascada de circuitos comparadores añadiendo entradas. (Será prioritaria la salida del comparador MSB)



C. g. → Comparador para n° de 2 bits

a ₁	a ₀	b ₁	b ₀	G	L
0	0	0	0	0	0 → igual
0	0	0	1	0	1 → > X
0	0	1	0	1	0 → < X
0	0	1	1	0	0 → igual
0	1	0	0	0	1
0	1	0	1	0	1
0	1	1	0	0	1
0	1	1	1	0	1
1	0	0	0	1	0
1	0	0	1	1	0
1	0	1	0	1	0
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	1	0	0
1	1	1	0	0	0
1	1	1	1	0	0

NOTA: Para rellenar G y L solo hace falta Comparar a₁ b₁ y a₀ b₀, recordando la tabla de antes. Si no también se puede operar.

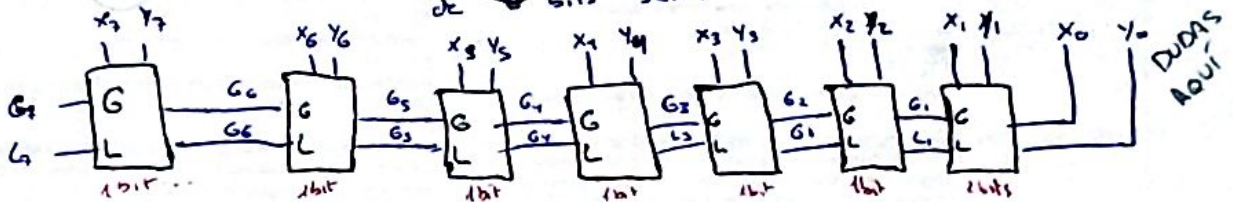
• Tras realizar Karnaugh para G y L obteniamos las siguientes expresiones:

$$G = A_1 \bar{B}_1 + A_1 A_0 \bar{B}_0 + \bar{B}_1 A_0 B_0$$

$$L = \bar{A}_1 B_1 + \bar{A}_1 \bar{A}_0 B_0 + B_1 \bar{A}_0 B_0$$

• Apartir de estas creamos el circuito (G y L son salidas).

NOTA Se pueden crear comparadores de n° de + bits con módulos comparadores básicos de 2 bits. Un ejemplo de n° de bits seria:



Un decodificador posee n entradas y hasta 2ⁿ salidas. Detecta una determinada combinación de bits (código). Cada salida indica si está presente cada una de las posibles combinaciones.

E.g

↳ Un decodificador de 2 entradas para 4 salidas.

↳ La salida 3 (11₁₀) del decu se activa cuando las entradas tienen valor 3 (11₁₀)

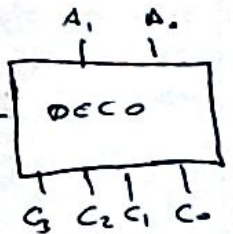


↳ Activación a nivel alto.

Para indicar cuándo aparece una combinación en las entradas de un circuito por medio de un nivel alto en la salida se emplean puertas AND, pues solo se activa si sus entradas son 1.

↳ Entrada de habilitación: Permite activar o desactivar el decodificador sin tener que modificar sus entradas. C

Si C = 0 → Deco desactivado → Todas salidas vale 0.
Si C = 1 → Deco activo → la salida correspondiente a la combinación de entradas vale 1. El resto 0.



↳ Activación a nivel bajo

En vez de usar AND se usa **NOR**.

→ Generación de funciones lógicas.

Cada salida del decodificador se corresponde con un término producto o con un término suma, podemos usar un decodificador para implementar una función lógica.

C.G. Dada una tabla de verdad, obtener la función como suma de productos o como producto de sumas

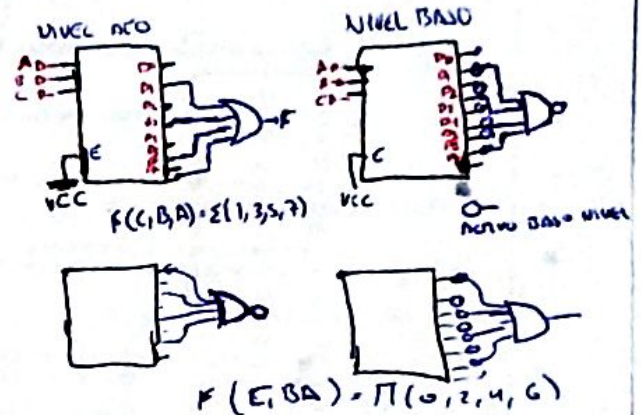
	C	B	A	F
0)	0	0	0	0
1)	0	0	1	1
2)	0	1	0	0
3)	0	1	1	1
4)	1	0	0	0
5)	1	0	1	1
6)	1	1	0	0
7)	1	1	1	1

$$\rightarrow F(C, B, A) = \Sigma(1, 3, 5, 7)$$

$$\rightarrow F(C, B, A) = \Pi(0, 2, 4, 6)$$

Las Sumas de productos se pueden expresar con puertas OR o con puertas NAND (ACTIVO NIVEL ALTO)

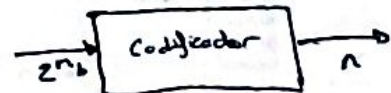
El Producto de sumas se pueden expresar con puertas AND o puertas NOR (ACTIVO NIVEL BAJO)



Codificador hace lo contrario a un decodificador: Cuando se activa una entrada, genera una salida codificada correspondiente a la entrada.

Tiene n salidas y 2^n entradas. La salida es la codificación binaria de la entrada activa.

C.G. Codificador de 4 entradas y 2 salidas.



La Cuando se activa la entrada 2, a la salida se muestra $10_{(2)}$ (representa el 2)

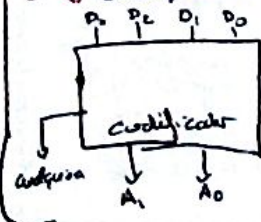
Un codificador activo a nivel alto generará una combinación de salida si la entrada correspondiente está a nivel alto.

Los Codificadores con prioridad

¿Qué pasa si en un codificador se activan varias líneas de entrada? Resultado no definido.

Por ello existen codificadores con prioridad que asignan un orden de prioridad a las entradas, si existen varias activas solo se tiene en cuenta la de mayor prioridad. Necesario añadir una salida (llamada cualquiera) que se active si alguna de las entradas está activa.

C.G. Codificador 4/2 con prioridad



$$A_0 = D_1 \cdot \overline{D_2} + D_3$$

$$A_1 = D_2 + D_3$$

$$\text{CUALQUERA} = D_0 + D_1 + D_2 + D_3$$

D_3	D_2	D_1	D_0	A_1	A_0	CUALQUERA
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

MULTIPLEXORES (SELECTORES)

Dirige la información procedente de varias Entradas hacia una única salida.

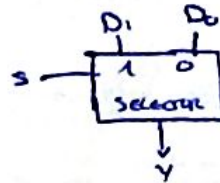
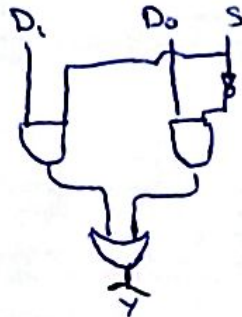
Posee 2^n entradas de datos, 1 salida de datos, n entradas de selección de canal de datos. (la entrada de selección indica que entrada se conecta a la salida).

SELECCIÓN /
DISTRIBUCIÓN
DE DATOS
(MULTIPLEXOR/
DEMULTIPLEXOR)

C.g. Multiplexor con 2 entradas y 1 de selección

S	Y
0	D_0
1	D_1

$$Y = \bar{S} \cdot D_0 + S \cdot D_1$$



• Funciones lógicas: Para generar funciones lógicas de un multiplexor (MUX):

- Las entradas de selección representan las variables de la función
- Las entradas de datos representan las posibles salidas según la tabla.

→ Procedimiento:

- Tabla de verdad: Se parte de la función lógica que quieres implementar
- Entradas de selección: se conectan directamente a las variables de entrada de la función
- Las combinaciones cuya salida = 1 → se conecta la entrada de datos a 1 (VCC).
Las combinaciones cuya salida = 0 → se conecta la entrada de datos a 0 (GND).

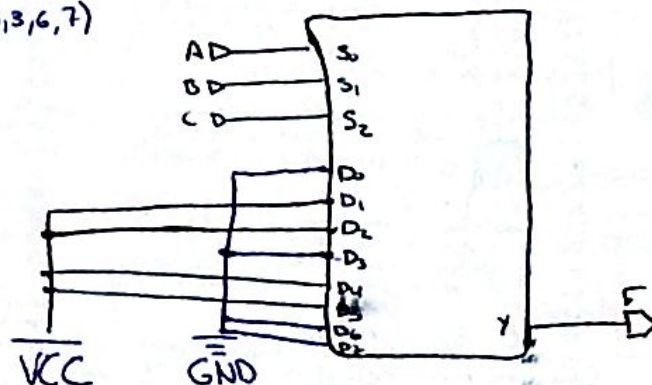
NOTA: IMPORTANTE respetar el orden de conexión (si $S_0 = 00, 01, 10, 11$)

C.g. SEGUICILLO → VARIABLES = ENTRADAS SELECCIÓN

Mux de 3 entradas de selección

$$F(C, B, A) = \Sigma(1, 2, 4, 6) ; F(C, B, A) = \Pi(0, 3, 5, 7)$$

	C	B	A	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	1
3	0	1	1	0
4	1	0	0	1
5	1	0	1	0
6	1	1	0	1
7	1	1	1	0

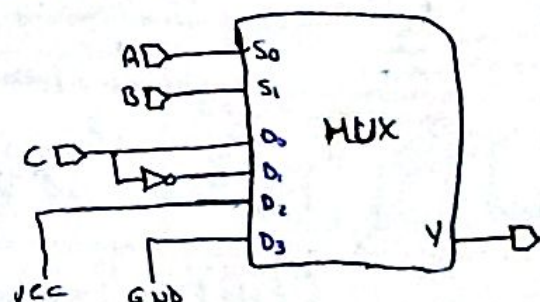


C.g. HARDER → + VARIABLES - ENTRADAS DE SELECCIÓN → 3 / 2 (USAMOS TABLA KARNAUGH)

$$F(C, B, A) = \Sigma(1, 2, 4, 6) ; F(C, B, A) = \Pi(0, 3, 5, 7)$$

C	B	A	D0	D1	D2	D3
00	0	0	0	0	0	0
01	0	1	0	0	0	0
10	1	0	0	0	0	0
11	1	1	0	0	0	0

- Si la combinación es 00 → GND
- Si la combinación es 11 → VCC
- Si no, C = 1 se conecta C
- Si no, C = 0 se conecta $\bar{C}$



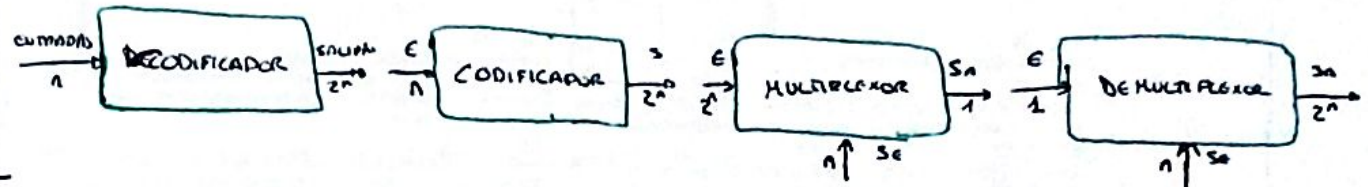
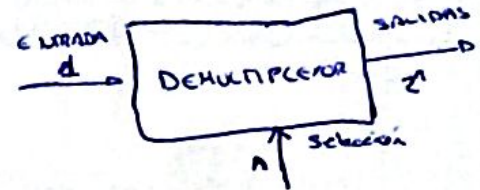
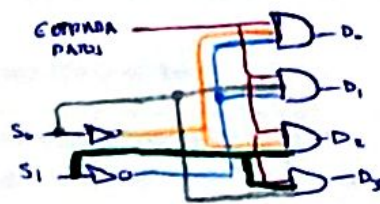
DEMÚLTIPLEXORES (DISTRIBUIDORES)

Lo contrario a un multiplexor. Distribuye los datos de una entrada entre varias salidas.

Posee 2^n salidas de datos, 1 entrada de datos y n entradas de selección.

Ej. Demultiplexor 4 salidas, 2 selección

S_1	S_0	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1



SISTEMAS SECUENCIALES

A diferencia de los sistemas combinacionales, que se calculan exclusivamente por los valores de entrada y son incapaces de recordar lo que ha ocurrido en el pasado, los circuitos secuenciales calculan sus salidas en base al estado del sistema y los valores actuales de entrada, ergo, tienen memoria.



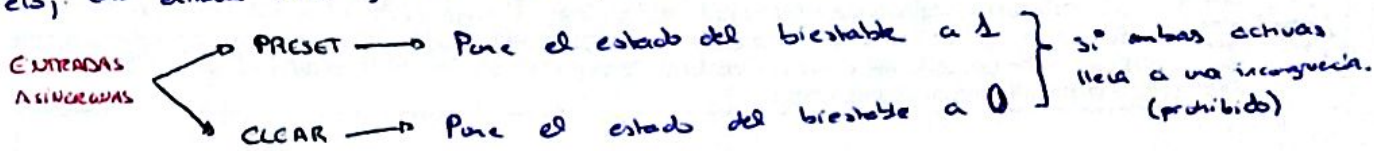
BICESTABLES

NOMBRE DEL BICESTABLE	SÍMBOLO DEL BICESTABLE	TABLA CARACTERÍSTICA	ECUACION CARACTERÍSTICA	TABLA DE EXITACION	DIAGRAMA DE CAMBIOS DE ESTADO	CIRCUITO																														
SR		<table border="1"> <tr> <th>S</th> <th>R</th> <th>Q^+</th> </tr> <tr> <td>0</td> <td>0</td> <td>Q</td> </tr> <tr> <td>0</td> <td>1</td> <td>0</td> </tr> <tr> <td>1</td> <td>0</td> <td>1</td> </tr> <tr> <td>1</td> <td>1</td> <td>Q</td> </tr> </table>	S	R	Q^+	0	0	Q	0	1	0	1	0	1	1	1	Q	$Q^+ = S + \bar{R}Q$	<table border="1"> <tr> <th>Q</th> <th>Q^+</th> <th>SR</th> </tr> <tr> <td>0</td> <td>0</td> <td>0 X</td> </tr> <tr> <td>0</td> <td>1</td> <td>1 0</td> </tr> <tr> <td>1</td> <td>0</td> <td>0 1</td> </tr> <tr> <td>1</td> <td>1</td> <td>X 0</td> </tr> </table>	Q	Q^+	SR	0	0	0 X	0	1	1 0	1	0	0 1	1	1	X 0		
S	R	Q^+																																		
0	0	Q																																		
0	1	0																																		
1	0	1																																		
1	1	Q																																		
Q	Q^+	SR																																		
0	0	0 X																																		
0	1	1 0																																		
1	0	0 1																																		
1	1	X 0																																		
JK		<table border="1"> <tr> <th>J</th> <th>K</th> <th>Q^+</th> </tr> <tr> <td>0</td> <td>0</td> <td>Q</td> </tr> <tr> <td>0</td> <td>1</td> <td>0</td> </tr> <tr> <td>1</td> <td>0</td> <td>1</td> </tr> <tr> <td>1</td> <td>1</td> <td>$\bar{Q}$</td> </tr> </table>	J	K	Q^+	0	0	Q	0	1	0	1	0	1	1	1	$\bar{Q}$	$Q^+ = J\bar{Q} + \bar{K}Q$	<table border="1"> <tr> <th>Q</th> <th>Q^+</th> <th>JK</th> </tr> <tr> <td>0</td> <td>0</td> <td>0 X</td> </tr> <tr> <td>0</td> <td>1</td> <td>1 X</td> </tr> <tr> <td>1</td> <td>0</td> <td>X 1</td> </tr> <tr> <td>1</td> <td>1</td> <td>X 0</td> </tr> </table>	Q	Q^+	JK	0	0	0 X	0	1	1 X	1	0	X 1	1	1	X 0		
J	K	Q^+																																		
0	0	Q																																		
0	1	0																																		
1	0	1																																		
1	1	$\bar{Q}$																																		
Q	Q^+	JK																																		
0	0	0 X																																		
0	1	1 X																																		
1	0	X 1																																		
1	1	X 0																																		
D		<table border="1"> <tr> <th>D</th> <th>Q^+</th> </tr> <tr> <td>0</td> <td>0</td> </tr> <tr> <td>1</td> <td>1</td> </tr> </table>	D	Q^+	0	0	1	1	$Q^+ = D$	<table border="1"> <tr> <th>Q</th> <th>Q^+</th> <th>D</th> </tr> <tr> <td>0</td> <td>0</td> <td>0</td> </tr> <tr> <td>0</td> <td>1</td> <td>1</td> </tr> <tr> <td>1</td> <td>0</td> <td>0</td> </tr> <tr> <td>1</td> <td>1</td> <td>1</td> </tr> </table>	Q	Q^+	D	0	0	0	0	1	1	1	0	0	1	1	1											
D	Q^+																																			
0	0																																			
1	1																																			
Q	Q^+	D																																		
0	0	0																																		
0	1	1																																		
1	0	0																																		
1	1	1																																		
T		<table border="1"> <tr> <th>T</th> <th>Q^+</th> </tr> <tr> <td>0</td> <td>Q</td> </tr> <tr> <td>1</td> <td>$\bar{Q}$</td> </tr> </table>	T	Q^+	0	Q	1	$\bar{Q}$	$Q^+ = T\bar{Q} + \bar{T}Q$	<table border="1"> <tr> <th>Q</th> <th>Q^+</th> <th>T</th> </tr> <tr> <td>0</td> <td>0</td> <td>0</td> </tr> <tr> <td>0</td> <td>1</td> <td>1</td> </tr> <tr> <td>1</td> <td>0</td> <td>1</td> </tr> <tr> <td>1</td> <td>1</td> <td>0</td> </tr> </table>	Q	Q^+	T	0	0	0	0	1	1	1	0	1	1	1	0											
T	Q^+																																			
0	Q																																			
1	$\bar{Q}$																																			
Q	Q^+	T																																		
0	0	0																																		
0	1	1																																		
1	0	1																																		
1	1	0																																		

TODOS ESTÁN
SINCRONIZADOS
AUNQUE PRE Y CLR SON ASÍNCRONOS

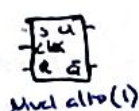
• ENTRADAS ASÍNCRONAS

Los biestables pueden disponer de entradas asíncronas independientes de la señal de reloj. Las entradas **Preset** y **Clear** tienen prioridad sobre el resto.



• BIESTABLES ACTIVOS POR FLANCO

↳ Activación por niveles: los biestables activos por nivel están habilitados siempre que la señal de reloj del sistema está en un nivel determinado ya sea 0 ó 1.



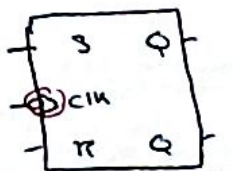
Este método produce un problema, pues durante el tiempo que dure el nivel, el valor del biestable puede cambiar tantas veces como cambie la entrada.

↳ Activación por flanco: otra técnica (usada) es que los biestables se activen en los flancos de una señal de reloj, ya sea los de subida o bajada.

↳ Biestables maestro-esclavo: combina 2 biestables

- La entrada al biestable maestro es la entrada del circuito
- La salida del biestable maestro es la entrada del esclavo
- La salida del esclavo es la salida del circuito

Ambos biestables se sincronizan con una señal reloj, mas el maestro se habilita a 0 y el esclavo a 1.



NOTA: el triángulo a los del CLK, indica que es activado por flanco

MODELOS DE MÁQUINAS DE ESTADOS FINITOS

Como se vio anteriormente, los sist. sec. se caracterizan por recordar el estado anterior. Para recordar este es necesario recordar las entradas anteriores al sistema.

Por ende, el estado de un sist. sec. depende de las **entradas del circuito (actuales)** y el **estado anterior (estado anterior)**.

• Los **diagramas de estados** muestran todos los estados posibles del sistema y las transiciones entre ellos

• Máquinas de estado finito: esta forma de caracterizar un sist. sec. está basada en un modelo de máquinas de estado finito (FSM - Finite-state Machine).
Teóricamente definimos el modelo FSM como una sextupla:

$$D \equiv \{E, S, N, \mu, \lambda, \lambda_0\}$$

→ E → entradas (I-inputs)

→ S → salidas (O-outputs)

→ N → conjunto estados (S-states)

→ μ → estado siguiente

→ λ → función de salida

→ λ_0 → estado inicial

• función salida λ :

Define valores de salida en base al estado y entradas actuales. salida $\lambda(x, E_c)$

• función estado siguiente μ :
Define estado siguiente en base estado y entradas actuales. $\mu(x, E_c)$

ANÁLISIS
Y
DISEÑO
DE
SISTEMAS
SECUENCIALES

(1)

• TIPOS DE FSM

Dependiendo de como se define la función de salida, encontramos 2 tipos de FSM:

↳ **Modelo Moore:** describe sist. sec. cuyas salidas dependen únicamente del estado del circuito $\equiv$ entrada

$E_t \equiv$ estado

$x_t \equiv$ entradas

$$Salida = Y_t = \lambda(E_t)$$

$$E_{t+1} = \mu(X_t, E_t)$$

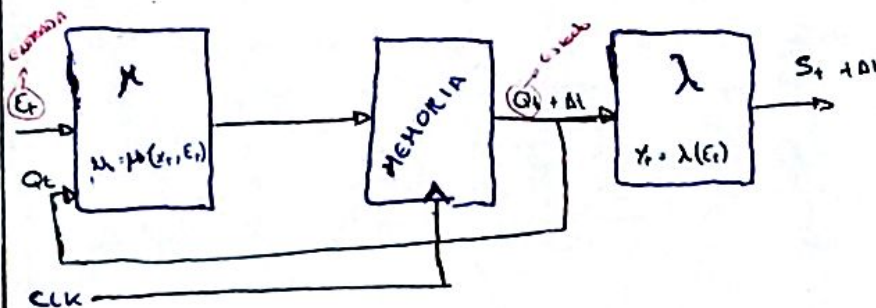
↳ **Modelo Mealy:** describe sist. sec. cuyas salidas dependen del estado del circuito y de las entradas en el intervalo de tiempo actual.

$$Y_t = \lambda(E_t, X_t)$$

$$E_{t+1} = \mu(X_t, E_t)$$

Nota: El estado del sistema sec. se almacena en circuitos biestables que actúan como elementos de memorización.

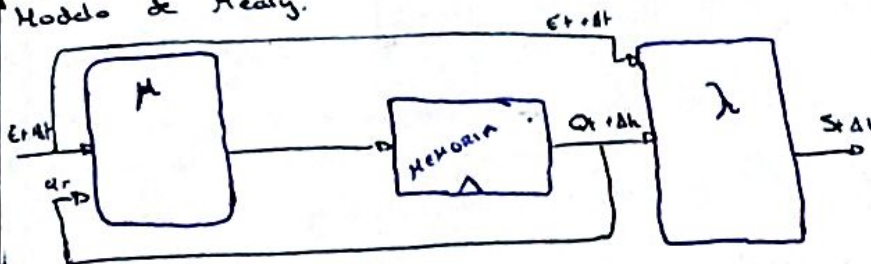
Modelo de Moore



$$\left\{ \begin{array}{l} Q_{t+1} = \mu(E_t, X_t) \\ Y_t = \lambda(Q_t) \end{array} \right.$$

Ecuaciones de MOORE

Modelo de Mealy.



$$\left\{ \begin{array}{l} Q_{t+1} = \mu(E_t, X_t) \\ Y_t = \lambda(Q_t, X_t) \end{array} \right.$$

Ecuaciones de MEALY

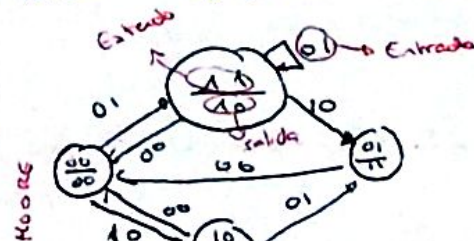
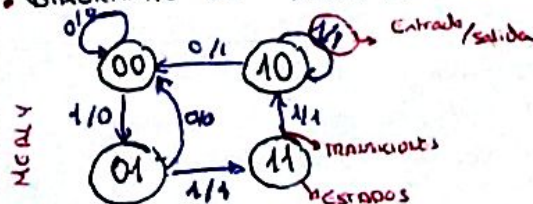
Procedimiento de análisis de sistemas tipo Moore/Mealy.

- Obtener ecuaciones de excitación
Ej: $D_0 = C_0 \oplus Q_0$
 $D_1 = Q_0 \oplus Q_1 + C_1$
- Usar tabla excitación obtener ecuaciones de estado sig. y de salida
Ej: $Q_0^+ = ?$
 $Q_1^+ = ?$
- Generar tablas de estado sig. y de salida
- Generar diagramas de estado
- Desarrollar diagramas de tiempo
- Simular el circuito lógico

Procedimiento de diseño de sistemas secuenciales (AFB)

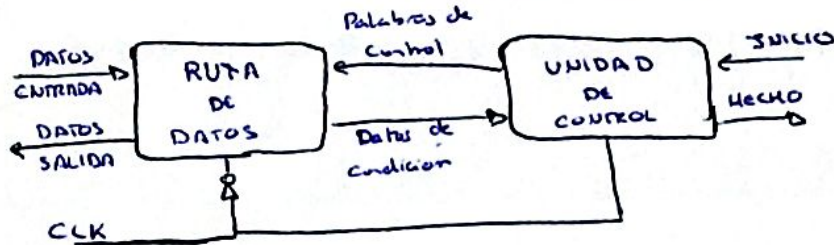
- Desarrollo diagrama de estados
- Tablas de estados sig. y salidas
- Minimizar estados (Karnaugh)
- Codificar estados sig. y salidas
- obtener ec. de estado sig. y salidas
- Elegir elementos biestables
- obtener esquema lógico y de tiempo

DIAGRAMAS DE MEALY / MOORE



Estados	Q_1	Q_0
00	0	0
01	0	1
10	1	0
11	1	1

2 bits
↓
2 biestables.

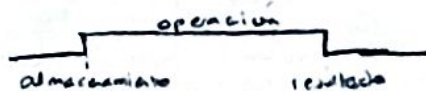


NOTA: Esquema genérico de un sistema complejo.

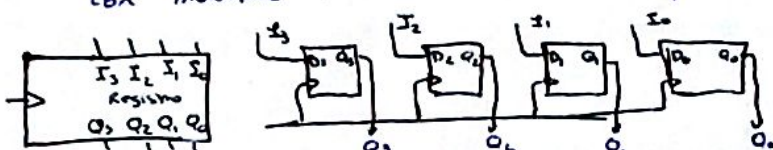
CAMINO DE DATOS

Los caminos de datos son sistemas digitales que se utilizan para la manipulación de datos y realización de cálculos numéricos. Funcionan de la siguiente manera:

- Los valores de variables y constantes se almacenan en registros o memorias en el flanco de la señal de reloj.
- Estos valores son modificados durante el periodo entre 2 flancos de reloj por medio de lógica combinatoria.
- Los resultados de estas operaciones se vuelven a almacenar en los registros o memorias en el siguiente flanco de reloj.



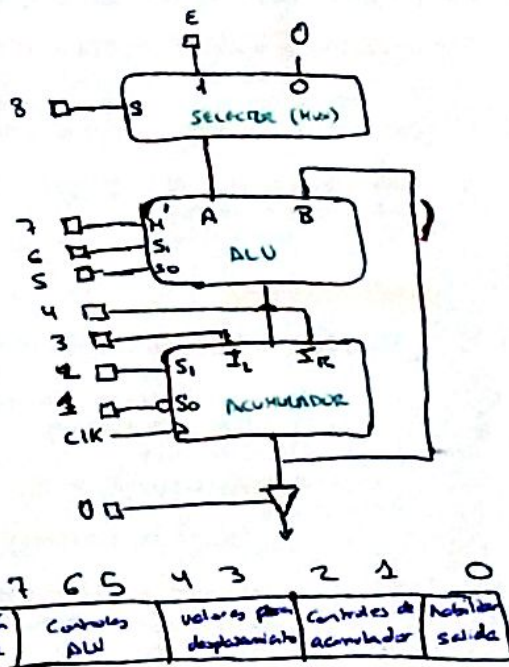
El Registro es un circuito que permite almacenar múltiples bits. Se conforman con múltiples biestables conectados en paralelo. En su forma más simple solo biestables.



Un registro más complejo, además de almacenar datos, permite realizar operaciones con estos.

CAMINO DE DATOS SENCILLO

- El multiplexor (selector), permite elegir el operando A de la ALU entre un 0 y el valor de entrada.
- El operando B de la ALU será siempre el contenido del acumulador. **operaciones ALU***
- El acumulador es un registro de desplazamiento con carga en paralelo. El contenido de este pasa a la salida a través de un adaptador inversor.



En cada ciclo del funcionamiento será necesario determinar los valores de las 9 señales de control. Estos valores son una palabra de control (código) de 9 bits.



CAMINO DE DATOS COMPLEJO

Para operaciones aritméticas + complejas, necesitamos un camino de datos + elaborado. Para almacenar + de 1 variable temporal es necesario usar un banco de registros en lugar de un acumulador.

El **SINCRONIZADO** Banco de registros: circuito que permite almacenar varios datos, cada uno en un registro, se construyen de forma que a cada ciclo se pueda leer 2 registros y escribir un registro.



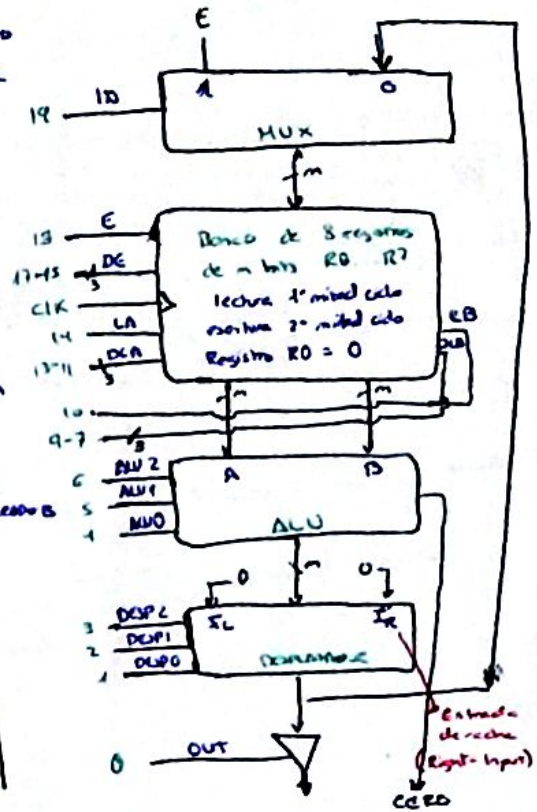
- El banco de registros de 3 puertos se encarga tanto de proporcionar los 2 operandos de la ALU como de memorizar el resultado de la operación

- Este camino de datos se controla con 20 bits que determinan los destinos, fuentes y operaciones del circuito

- A continuación adjuntamos las tablas de operaciones de la ALU y el de registro de desplazamiento



19	10	Selección control
18	E	habilitación de escritura
17	DE2	dirección escritura
16	DE1	
15	DE0	
14	LA	habilitación de lectura
13	DLA2	dirección de lectura (operando A)
12	DLA1	
11	DLA0	
10	LB	habilitación lectura
9	DLB2	dirección de lectura (operando B)
8	DLB1	
7	DLB0	
6	ALU2	operación de la ALU
5	ALU1	
4	ALU0	
3	DEPL2	operación del desplazador
2	DEPL1	
1	DEPL0	
0	OUT	habilitación de salida



* OPERACIONES DE LA ALU

		ALU2	ALU1	ALU0
Complementar	no A	0	0	0
AND	A and B	0	0	1
XOR	A xor B	0	1	0
OR	A or B	0	1	1
Decrementar	A - 1	1	0	0
Sumar	A + B	1	0	1
Restar	A - B	1	1	0
Incrementar	A + 1	1	1	1

OPERACIONES DESPLAZADOR

	DEPL2	DEPL1	DEPL0
Guardar el valor	0	0	0
Pasar el valor	0	0	1
No usada	0	1	0
No usada	0	1	1
Desplazar a la izq	1	0	0
Rotar a la izq	1	0	1
Desplazar a la dch	1	1	0
Rotar a la dch	1	1	1

UNIDAD DE CONTROL

La unidad de control es un circuito encargado de proporcionar la palabra de control adecuada en cada ciclo de reloj al camino de datos. Esta se puede implementar como un circuito secuencial con biestables que almacenen su estado, una lógica de estado siguiente y una lógica de salida, proporcionando así la palabra de control.

- Diseño de unidad de control (APS)

- Definir para qué se va a usar (contador, etc)
- Asignar variables a la registros del banco de registros
- Inicializar variables y aplicar algoritmo
- Para diseñar la U.C. recordamos usar un diagrama de estados

E.g. U.C. para contador de unos

- Recibe un dato de entrada
- Conta el nº de 1 que contiene
- Lo representa en la salida.

* Suponemos que el dato de entrada proviene de exterior y está disponible en la entrada del circuito.

Tabla con la Lógica de Salida y las señales de la palabra de control.

Crear circuito

- Utilizamos 4 variables temporales asignadas a 4 registros del banco de registros.

- R₁: Dato
- R₂: Máscara
- R₃: Cuenta
- R₄: Temp

INICIALIZACIÓN
Dato = Input
Cuenta = 0
Máscara = 1

ALGORITMO
while Dato < 0 repeat
Temp = Dato AND Máscara
Cuenta = Cuenta + Temp
Dato = Dato >> 1
end while

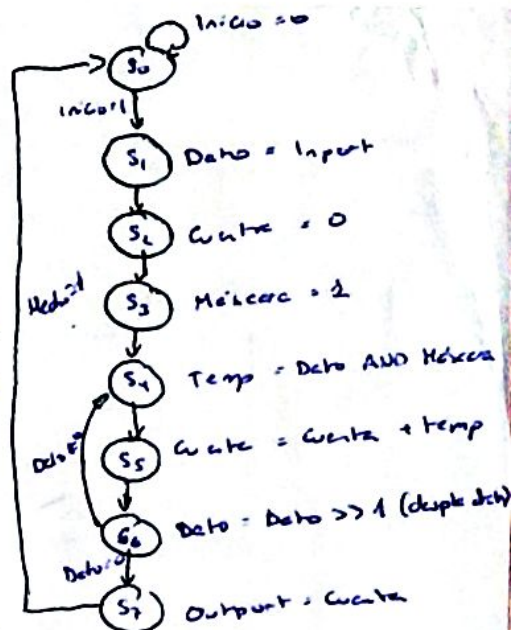
FINAL
Output = Cuenta

CAMINO DE DATOS
Y
UNIDAD DE CONTROL

- La unidad de control será un módulo secuencial con 2 señales de entrada y 21 de salida: 20 al camino de datos y 1 a del exterior

- Inicio indica cuando comenzar la cuenta
- Dato $\neq 0$ Señal de entrada proporcionada por el datapath
- Hecho es señal que indica que la cuenta ha finalizado y se encuentra a la salida.

Q ₂	Q ₁	Q ₀	Palabra control	IE	Dir. lectura	Dir. escritura	DTE	ALU	PERMISO	Hecho
0	0	0	0	0	X X	X	0	X X	X X	0
0	0	1	1	1	R ₁	X	X	X	X	0
0	1	0	2	0	R ₃	0	0	SUM	PASAR	0
0	1	1	3	0	R ₂	0	X	hacer	PASAR	0
1	0	0	4	0	R ₄	R ₁	R ₂	AND	PASAR	0
1	0	1	5	0	R ₃	R ₃	R ₄	SUMAR	PASAR	0
1	1	0	6	0	R ₁	R ₁	0	SUMAR	Dep del	0
1	1	1	7	0	X X X	R ₃	0	SUMAR	Pasar	1

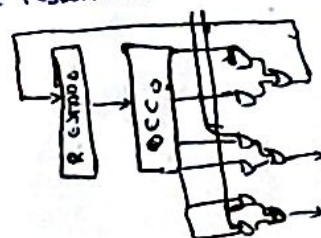


CAMINO DE
DATOS
Y
UNIDAD DE
CONTROL

- Implementación de la unidad de control.

Por ahora hemos diseñado la UC como un sist. sec. con biestables, lo que resulta sencillo para UCs sencillas. Pero para módulos de control con 4 estados es más rentable emplear otros diseños que simplifica el resultado

- Decodificador: Cada estado se identifica mediante una señal que valga 1 cuando el registro de estado esté en un estado concreto y 0 en los demás casos.



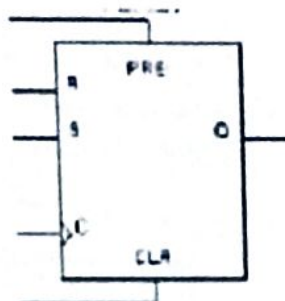
- Contador: en el caso de que la UC tenga secuencias de estados con un único sucesor se puede codificar los estados de secuencia consecutivamente y sustituir el registro de estado por un contador.

El contador incrementará el estado por seguir por una secuencia o cargará un nuevo valor para saltar a otra.

- ROM (control microprogramado): se sustituye la lógica de estado por una memoria ROM de control. El registro de estado actúa como un registro de direcciones para la ROM. El microprograma de la ROM determina el estado siguiente a partir del estado actual, ya sea seguir incrementando secuencialmente o realizar un salto a otra secuencia. (se usará para diseño de procesadores complejos).

Nombre del biestable	Símbolo del biestable	Tabla característica	Ecuación característica	Tabla de excitación
----------------------	-----------------------	----------------------	-------------------------	---------------------

SR

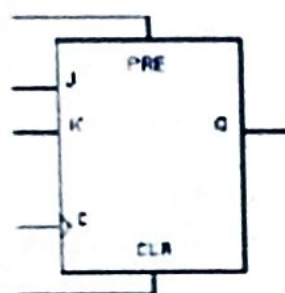


S	R	Q(t+1)
0	0	Q(t)
0	1	0
1	0	1
1	1	X

$$Q(t+1) = S + R'Q$$

Q(t)	Q(t+1)	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

JK

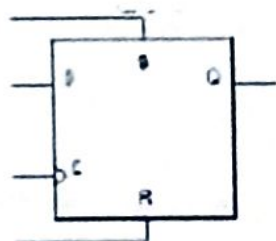


J	K	Q(t+1)
0	0	Q(t)
0	1	0
1	0	1
1	1	Q'(t)

$$Q(t+1) = JQ' + K'Q$$

Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

D

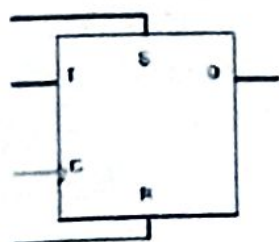


D	Q(t+1)
0	0
1	1

$$Q(t+1) = D$$

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

T



T	Q(t+1)
0	Q(t)
1	Q'(t)

$$Q(t+1) = TQ' + T'Q$$

Q(t)	Q(t+1)	T
0	0	0
0	1	1
1	0	1
1	1	0